

# L3A: Higher-order Functions; Scope of Names

---

CS1101S: Programming Methodology

Boyd Anderson

26th August, 2026

Announcements

Nested declaration assignments

More fun with recursion

Higher-order functions

Scope of names

# Announcements

---

- ▶ Quests: they are “extra homework”! If you are struggling with your time management or work/life balance: Don't touch the quests!
- ▶ Contest “Beautifully Built” opens today!
- ▶ **Forum Hours.** Every Tuesday 6:30pm-8:30pm. Venue is COM1 basement.
- ▶ Reading Assessment 1: Read our announcement carefully. Sample questions have been uploaded.

# Some Words of Advice

---

- ▶ Read the textbook
- ▶ Use the substitution model (and stepper tool, if needed)
- ▶ Think, then program
- ▶ Less is more
- ▶ Use AI smartly (to optimise your learning)

Announcements

**Nested declaration assignments**

More fun with recursion

Higher-order functions

Scope of names

# Names can refer to intermediate values

---

Example from SICPy 1.3.2

$$f(x, y) = x(1 + xy)^2 + y(1 - y) + (1 + xy)(1 - y)$$

Compute  $f(2, 3)$

```
def f(x, y):  
    return (x * square(1 + x * y)  
            + y * (1 - y)  
            + (1 + x * y) * (1 - y))  
  
print(f(2, 3))
```



# Names can refer to intermediate values

---

Example from SICPy 1.3.2

$$f(x, y) = x(1 + xy)^2 + y(1 - y) + (1 + xy)(1 - y)$$

Compute  $f(2, 3)$

```
def f(x, y):  
    a = 1 + x * y  
    b = 1 - y  
    return x * square(a) + y * b + a * b  
  
print(f(2, 3))
```



Announcements

Nested declaration assignments

More fun with recursion

- Example: fractal runes (“Rune Reading”)

- Conditional statements (1.3.2)

- Example: coin change (1.2.2)

Higher-order functions

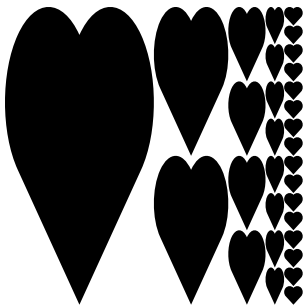
Scope of names



## Example from Mission “Rune Reading”: `fractal`

---

Define a function `fractal` that returns pictures like this:



when we call it like this: `fractal(heart, 5)`

# “Rune Reading” Example: fractal, Solution 1

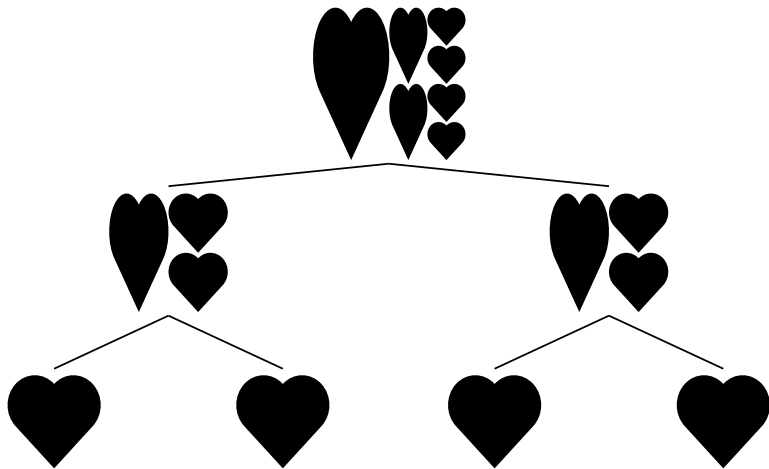
---

```
def fractal_1(rune, n):  
    return (rune if n == 1  
            else beside(rune,  
                        stack(fractal_1(rune, n - 1),  
                              fractal_1(rune, n - 1))))
```



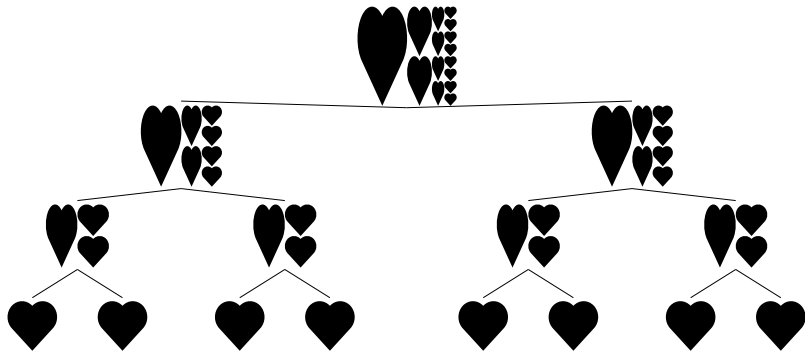
## The Call Tree for `fractal_1(heart, 3)`

---



## The Call Tree for `fractal_1(heart, 4)`

---



# Can we avoid tree recursion?

---

```
def fractal_1(rune, n):  
    return (rune if n == 1  
            else beside(rune,  
                        stack(fractal_1(rune, n - 1),  
                              fractal_1(rune, n - 1))))
```



## Question

Can we implement this function with linear recursion?

# Is this a good idea?

---

```
def fractal_2(rune, n):  
    smaller_frac = fractal_2(rune, n - 1)  
    return (rune if n == 1  
            else beside(rune,  
                        stack(smaller_frac, smaller_frac)))
```



Can we declare a name...

**...just for the alternative of the conditional?**

## Conditional statements (see SICPy 1.3.2)

---

```
def fractal_3(rune, n):  
    if n == 1:  
        return rune  
    else:  
        f = fractal_3(rune, n - 1)  
        return beside(rune, stack(f, f))
```



- ▶ Each branch of the conditional can have local declarations.
- ▶ A declaration assignment in a branch is only evaluated when the branch actually runs (more on scope shortly).
- ▶ Remember to return a result *in each branch*.  
(Otherwise `None` is returned.)

## Example: coin change (1.2.2)

---

- ▶ Given: Different kinds of coins (unlimited supply)
- ▶ Given: Amount of money in cents
- ▶ Wanted: Number of ways to change amount into coins

### Step 1

Read the problem *very carefully*



# Play

---

- ▶ Given: Different kinds of coins (unlimited supply)
- ▶ Given: Amount of money in cents
- ▶ Wanted: Number of ways to change amount into coins
- ▶ How about 10 cents?
- ▶ How about 20 cents?
- ▶ 5 cents?
- ▶ 0 cents?
- ▶ -1 cents?

## Step 2

Play with examples

# Think

---

- ▶ Example: Number of ways to change 120 cents
- ▶ Idea: Highest coin is either 100 or not 100
- ▶ In the first case: Smaller problem
- ▶ In the second case: Smaller problem
- ▶ Base cases?

## Step 3

Think: existing solution? divide-and-conquer? “wishful thinking”?

# Representing the kinds of coins

---

```
def first_denomination(kinds_of_coins):  
    return (5 if kinds_of_coins == 1  
            else 10 if kinds_of_coins == 2  
            else 20 if kinds_of_coins == 3  
            else 50 if kinds_of_coins == 4  
            else 100 if kinds_of_coins == 5  
            else 0)
```



# Idea in Python

---

```
def cc(amount, kinds_of_coins):  
    return (... # base cases  
        if ...  
        else cc(amount - first_denomination(kinds_of_coins),  
                kinds_of_coins)  
        + cc(amount, kinds_of_coins - 1))
```

## Step 4

Program using Python

# Adding base cases

---

```
def cc(amount, kinds_of_coins):  
    return (1 if amount == 0  
            else 0 if amount < 0 or kinds_of_coins == 0  
            else cc(amount -  
                    first_denomination(kinds_of_coins), kinds_of_coins)  
                + cc(amount, kinds_of_coins - 1))
```



## Step 5

Test your program

# The CS1101S 5-step Method of Problem Solving™

---

## Step 1

Read the problem *very carefully*

## Step 2

Play with examples

## Step 3

Think: existing solution? divide-and-conquer? "wishful thinking"?

## Step 4

Program using Python

## Step 5

Test your program

Announcements

Nested declaration assignments

More fun with recursion

Higher-order functions

- Functions as arguments (1.3.1)

- Lambda expressions (1.3.2)

- Functions as returned values

- Summary of new constructs today

Scope of names

# Passing functions to functions

---

```
def f(g, x):  
    return g(x)  
  
def g(y):  
    return y + 1  
  
print(f(g, 7))
```





# Passing more functions to functions

---

```
def f(g, x):  
    return g(g(x))  
  
def g(y):  
    return y + 1  
  
print(f(g, 7))
```



# We have seen this before!

---

Repeating the pattern  $n$  times

```
show(repeat_apply(make_cross, 4, rcross))
```



Combining and transforming pictures

```
def transform_mosaic(p1, p2, p3, p4, transform):  
    return transform(mosaic(p1, p2, p3, p4))
```

## Look at this function (1.3.1)

---

```
def sum_integers(a, b):  
    return 0 if a > b else a + sum_integers(a + 1, b)
```



## ...and this one

---

```
def cube(x):  
    return x * x * x  
  
def sum_skip_cubes(a, b):  
    return 0 if a > b else cube(a) + sum_skip_cubes(a + 2, b)
```



# Abstraction (1.3.1)

---

```
def sum(a, b):  
    return (0 if a > b  
           else ⟨compute value with a⟩  
           + sum(⟨next value from a⟩, b))
```

in Python:

```
def sum(term, a, next, b):  
    return 0 if a > b else term(a) + sum(term, next(a), next, b)
```



## sum\_integers **using** sum

---

```
def identity(x):  
    return x  
  
def plus_one(x):  
    return x + 1  
  
def sum_integers(a, b):  
    return sum(identity, a, plus_one, b)
```



## sum\_skip\_cubes **using** sum

---

```
def cube(x):  
    return x * x * x  
  
def plus_two(x):  
    return x + 2  
  
def sum_skip_cubes(a, b):  
    return sum(cube, a, plus_two, b)
```



## sum\_skip\_cubes **using** sum

---

```
def cube(x):  
    return x * x * x  
  
def plus_two(x):  
    return x + 2  
  
def sum_cubes(a, b):  
    return sum(cube, a, plus_two, b)
```



## Visibility

Can we “hide” cube and plus\_two inside of sum\_skip\_cubes?



# Yes, we can! (see also SICPy 1.1.8)

---

```
def sum_skip_cubes(a, b):  
    def cube(x):  
        return x * x * x  
    def plus_two(x):  
        return x + 2  
    return sum(cube, a, plus_two, b)
```



# Now You See It, Now You Don't

---

Let's tuck `fact_iter` in and see how many parameters it can shed along the way!

*Look for Check-In I3A in the Source Academy!*

## Another look at such local functions

---

```
def sum_skip_cubes(a, b):  
    def cube(x):  
        return x * x * x  
    def plus_two(x):  
        return x + 2  
    return sum(cube, a, plus_two, b)
```



This is still quite verbose

Do we need all these words such as `def`, `return`?

Do we need to even give names to these functions?

# No, we don't! Lambda expressions

---

```
def sum_skip_cubes(a, b):  
    def cube(x):  
        return x * x * x  
    def plus_two(x):  
        return x + 2  
    return sum(cube, a, plus_two, b)
```



```
# instead just write:  
def sum_skip_cubes(a, b):  
    return sum(lambda x: x * x * x, a, lambda x: x + 2, b)
```



## Lambda expressions (1.3.2)

---

New kind of expression

`lambda parameters : expression`

Meaning

The expression evaluates to a function value.

Function has given *parameters* and *expression* as its (only) return value;  
no `return` keyword needed.

$\lambda$

(yes, this is also why it's on our t-shirts)

# We can have more than one parameter

---

Comma-separated, just like a function definition

`lambda  $p_1$ ,  $p_2$ , \dots,  $p_n$  : expression`

```
add_three = lambda x, y, z: x + y + z  
print(add_three(1, 2, 3))
```



# An alternative syntax for function definition

---

```
def plus4(x):  
    return x + 4
```



can be written as

```
plus4 = lambda x: x + 4
```



# Returning Functions from Functions

---

```
def make_adder(x):  
    def adder(y):  
        return x + y  
    return adder  
  
adder_four = make_adder(4)  
print(adder_four(6))
```





## ...or with the new lambda expressions

---

```
def make_adder(x):  
    return lambda y: x + y  
  
adder_four = make_adder(4)  
print(adder_four(6))
```



# Returning Functions from Functions

---

```
def make_adder(x):  
    return lambda y: x + y
```

```
print((make_adder(4))(6))
```

```
# we can drop the parentheses:  
#  
# make_adder(4)(6)
```



# Returning Functions from Functions

---

```
def make_adder(x):  
    return lambda y: x + y  
  
adder_1 = make_adder(1)  
adder_2 = make_adder(2)  
  
# adder_1(10) returns 11  
print(adder_1(10))  
  
# adder_2(20) returns 22  
print(adder_2(20))
```



# Summary of new constructs today

---

- ▶ Nested declaration assignment and function definition statements
- ▶ Conditional statements
- ▶ Lambda expressions

Announcements

Nested declaration assignments

More fun with recursion

Higher-order functions

Scope of names

- Examples

- Overview of scoping rules

- The details

# Scope of names: an example

---

```
z = 2
def f(g):
    z = 4
    return g(z)

print(f(lambda y: y + z))
```



## Questions about scope

What names are declared by this program?

Which declaration does each name occurrence refer to?

# Scope of names: another example

---

```
x = 10
def square(x):
    return x * x

def addx(y):
    return y + x

print(square(x + 5) * addx(x + 20))
```



## Questions about scope

Which declaration does each occurrence of `x` refer to?

# Scope of names: yet another example

---

```
pi = 3.141592653589793

def circle_area_from_radius(r):
    pi = 22 / 7
    return pi * square(r)
```



## Questions about scope

Which declaration does the occurrence of `pi` refer to?



## Scope of names: hypotenuse example

---

```
def square(x):  
    return x * x  
  
def hypotenuse(a, b):  
    def sum_of_squares():  
        return square(a) + square(b)  
    return math.sqrt(sum_of_squares())
```



Names can refer to declarations outside of the immediately surrounding function definition.

# Overview of scoping rules

---

## Declarations mandatory

All names in Python must be declared.

## Forms of declaration

1. Primitive names
2. Declaration assignments
3. Parameters of function definitions and lambda expressions
4. Function name of function definitions

## Scoping rule

A name occurrence refers to the *closest surrounding declaration* of the name.

# Forms of Declaration

---

## (1) Primitive names

The Python §1 pages tell us what names are primitive, e.g. `math.floor`. We can also `import` further primitive names from modules. For example, from the `rune` module:

```
from rune import heart, quarter_turn_right
```

# Forms of Declaration

---

## (2) Declaration assignments

The scope of a declaration assignment is the body of the *immediately surrounding function*, or the whole program, if there is none.

A simple example (see SICPy 1.3.2)

```
z = 3
def f():
    print(z)
    z = 4
f()
```



A name declared in a function cannot be used before the declaration is fully evaluated, regardless of whether the same name is declared outside the function. Here, `print(z)` runs before the declaration `z = 4`.

# Forms of Declaration

---

## (2) Declaration assignments: what about branches?

```
z = 100
def f(x, y):
    if x > 0:
        z = x * y
    return z
print(f(3, 4))
```



The declaration assignment `z = x * y` is inside the `if`, but the local `z` still shadows the global one throughout `f`.

# Forms of Declaration

---

## (2) Declaration assignments: scope vs. value

$z$ 's *scope* covers all of  $f$ , but it only has a *value* once the assignment has actually run.

```
z = 100

def f(x, y):
    if x > 0:
        z = x * y
    return z

print(f(-3, 4))
```

$f(-3, 4)$  skips the `if`:  $z$  is still in scope but never gets a value (*not even the global  $z$  above*), so this is an error.

# Forms of Declaration

---

## (3) Parameters

The scope of the parameters of a lambda expression or function definition is the body of the function.

```
def f(x, y, z):  
    ... x ... y ... z ...  
  
lambda v, w, u: ... v ... w ... u ...
```

# Forms of Declaration

---

## (4) Function name

The scope of the function name of a function definition is as if the function was declared with a declaration assignment.

```
def f(x):  
    ...
```

as if we wrote

```
f = ...
```



# Lexical scoping

---

## Scoping rule

A name occurrence refers to the *closest surrounding declaration* of the name.

```
def f(x):  
    def g(w):  
        x = ...  
        y = x + 1  
        return lambda x: ...x...  
    return g(x)
```

# Important Ideas

---

- ▶ *Hiding* can be a useful abstraction technique
- ▶ *Recursion* is an elegant pattern of problem solving
- ▶ Functions can be *passed to* functions
- ▶ Functions can be *returned from* functions
- ▶ Higher-order functions are *useful* for building abstractions
- ▶ With nested functions, we need to understand the *scope* of names
- ▶ The CS1101S 5-step Method of Problem Solving™

# Concluding Unit 1

---

- ▶ Covered: textbook chapter 1
- ▶ Mental model: substitution model
- ▶ Big ideas: iterative/recursive processes, higher-order, scope
- ▶ Problem solving technique: recursion (“wishful thinking”)
- ▶ Assessment:
  - Reading Assessment 1, Friday Week 4
  - Programming Assessment 1, Friday Week 6
  - Big ideas: Mastery Check 1 (more later), before Friday Week 13
  - Missions/Quests: problem solving
- ▶ Friday: Curves and Sound
- ▶ Look out for Unit 2 with Prof Sanka: Building Abstractions with Data, starting with L4